SANJIVANI UNIVERSITY

SCHOOL OF ENGINEERING AND TECHNOLOGY

At: kopargaon, Dist:Ahmednagar, Maharashtra, India – 423601

DEPARTMENT (Artificial Intelligence and Machine Learning)

COURSE: DevOps and Cloud (IBM)

(24UAMAE201)

Academic Year 2026-2027

Mini Project Unit 3

NAME	**Vaishnavi Jadhav**
PRN NO	**2125UMLF1100**
SEM/YEAR	**2nd Year/3rd Sem**
BATCH	**SY(A)**

SUBMITTED TO

Mr. Karan Gaikwad Sir
(Professor)

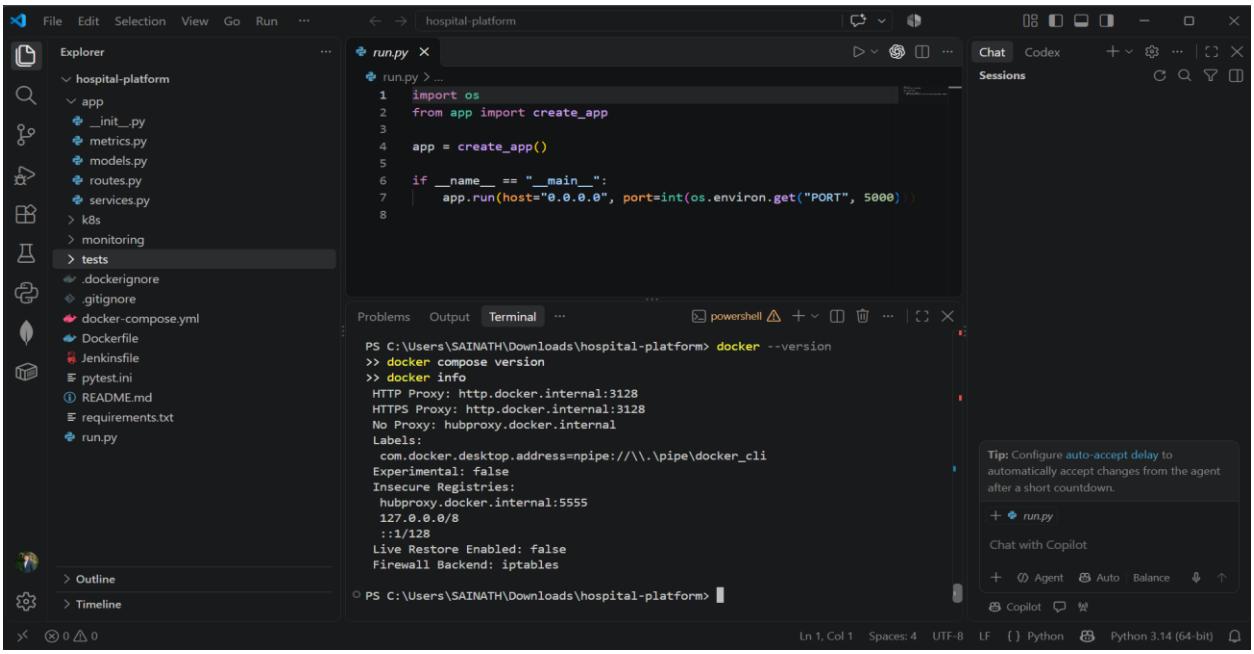
SUBMITTED BY

Vaishnavi Jadhav
(B.Tech 2nd year)

PRN: 2125UMLF1100

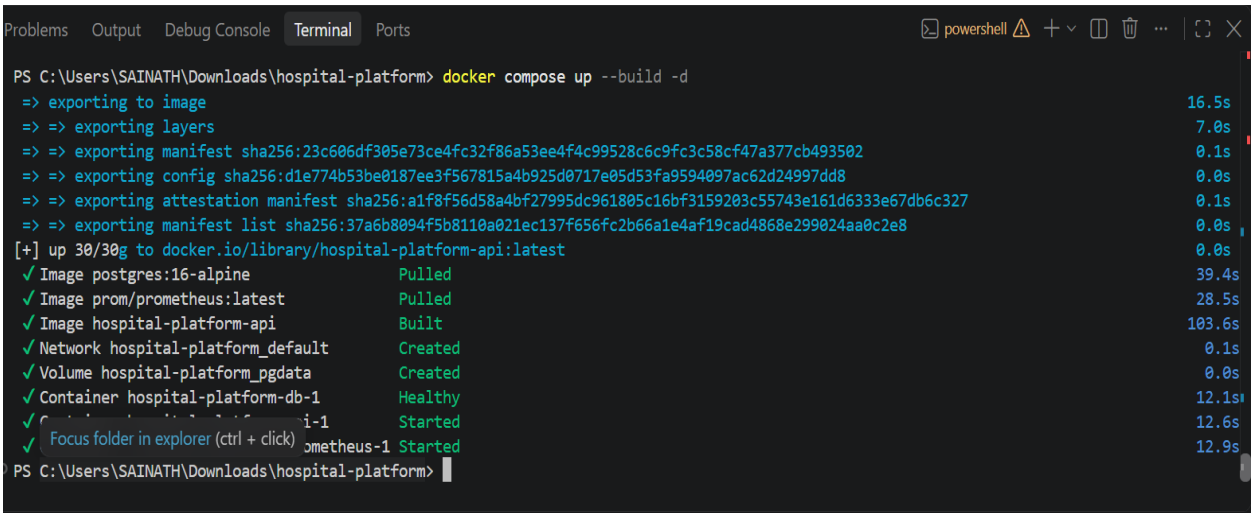
Step 1: Project Structure and Docker Environment Verification

- The Hospital Platform project was opened in Visual Studio Code, displaying the Flask application modules, Kubernetes and monitoring folders, test directory, Dockerfile, Docker Compose configuration, Jenkinsfile, and dependency files. In the PowerShell terminal, docker --version, docker compose version, and docker info were executed to verify the Docker installation and environment.



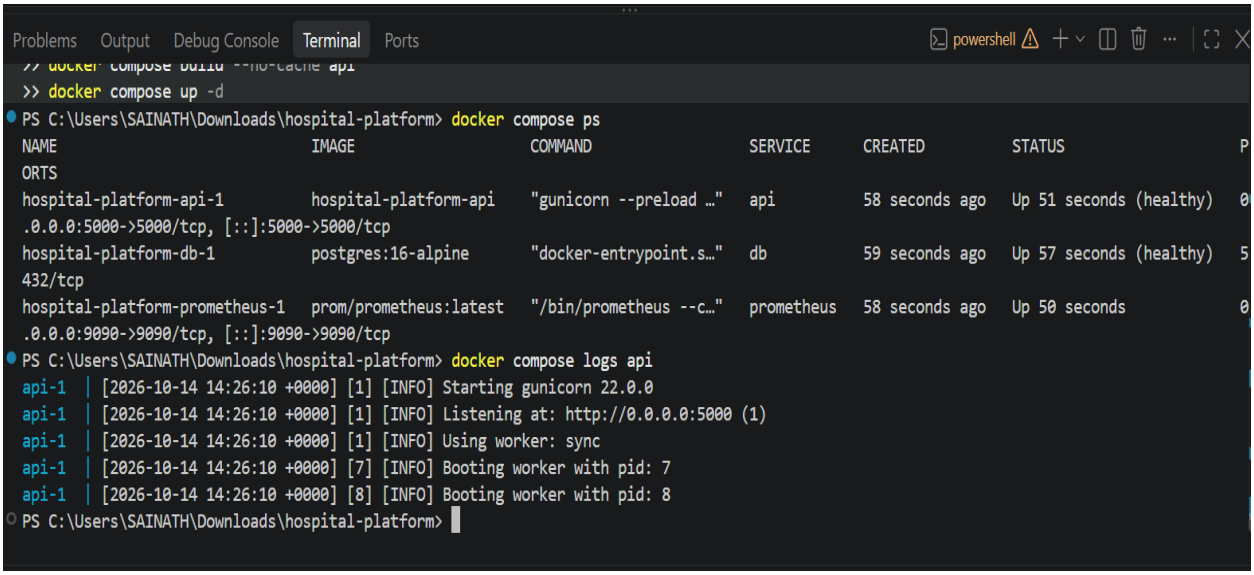
Step 2: Docker Compose Build and Multi-Container Deployment

- The command `docker compose up --build -d` was executed to build the Hospital Platform API image and start the application services in the background. The terminal confirms that the PostgreSQL image was pulled, the Prometheus image was pulled, the application image was built, the Docker network and persistent volume were created, and the database container became healthy while the application and Prometheus containers started.



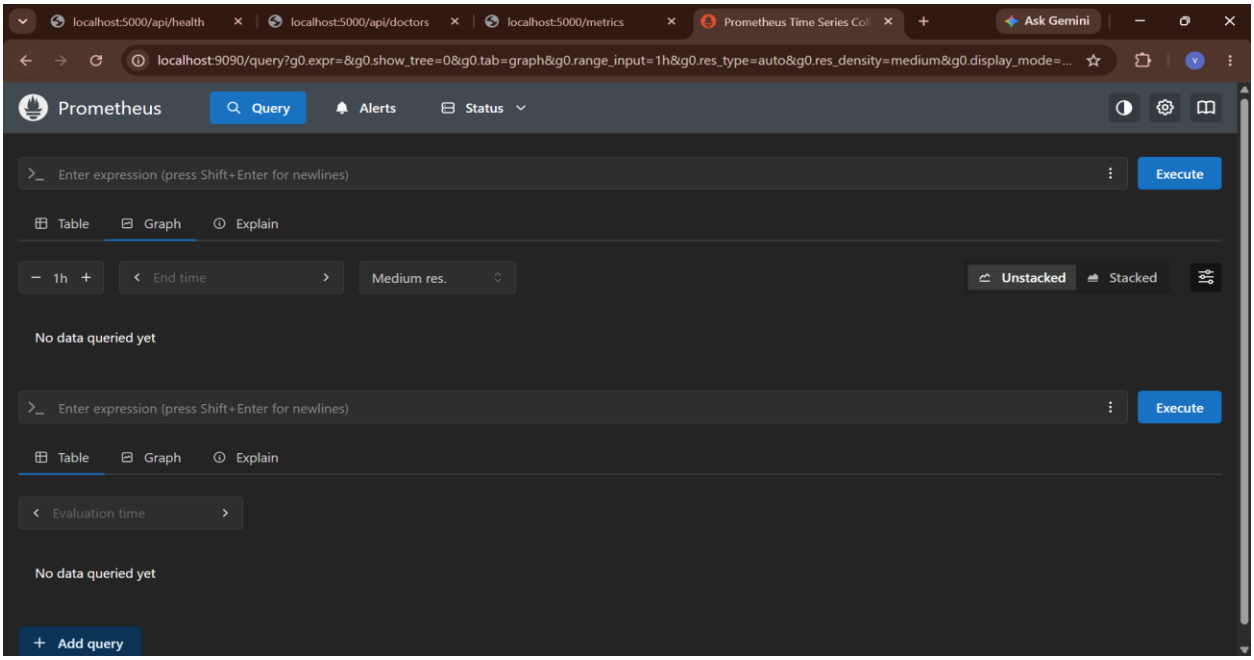
Step 3: Docker Container Status and Application Log Verification

- The `docker compose ps` command was used to verify the status of the API, PostgreSQL database, and Prometheus containers. The API and database containers were reported as healthy, while Prometheus was running. The `docker compose logs api` command confirmed that Gunicorn started successfully, listened on port `5000`, and initialized its worker processes.



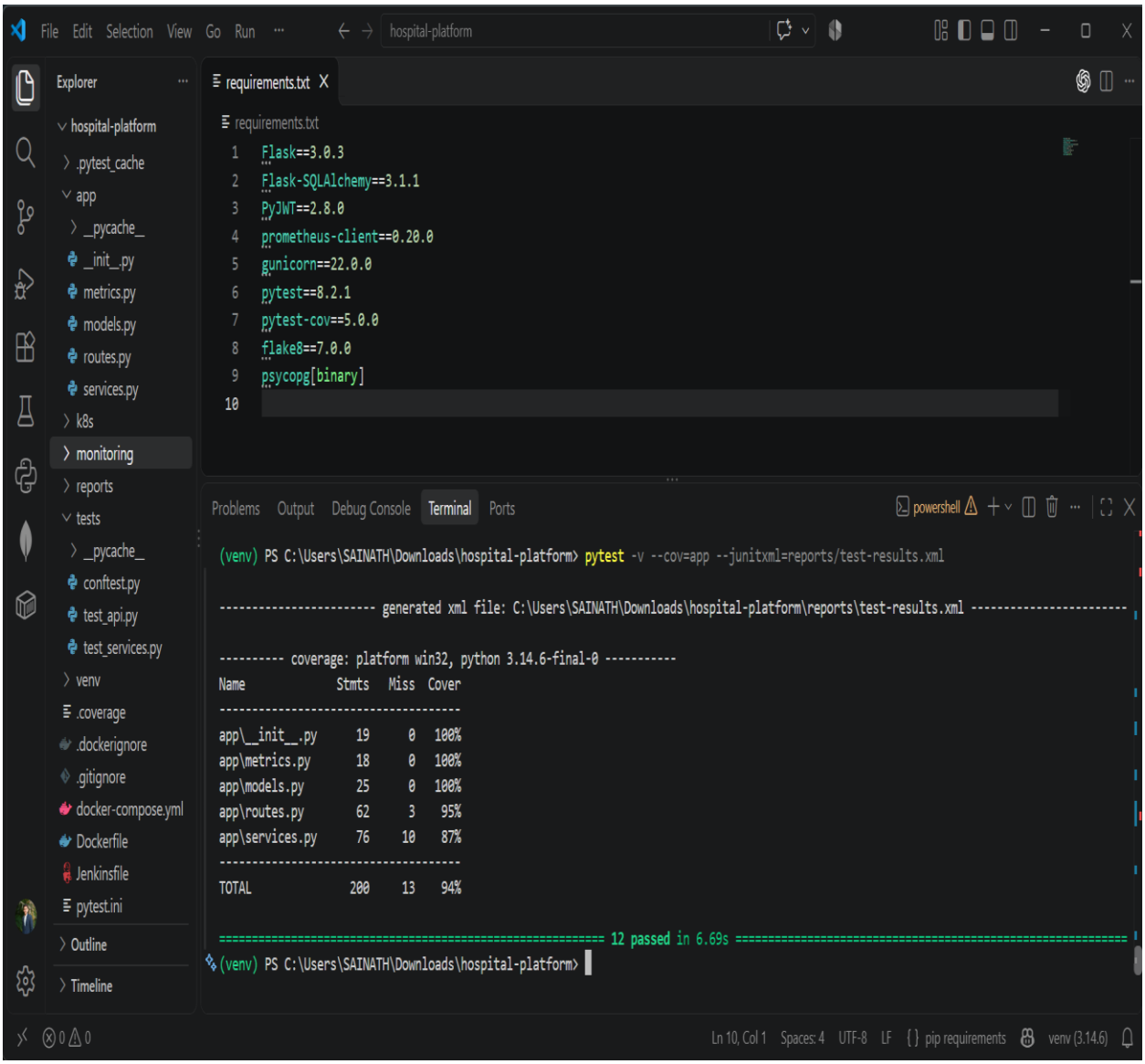
Step 4: API Health Check and Prometheus Monitoring Verification

- The application was tested through the browser using the `/api/health` endpoint, which returned an UP status for the Smart Hospital Platform. The `/api/doctors` endpoint returned doctor records with their specializations. The `/metrics` endpoint and Prometheus query interface were also opened to verify the monitoring setup; however, the Prometheus page showed that no query had been executed yet.



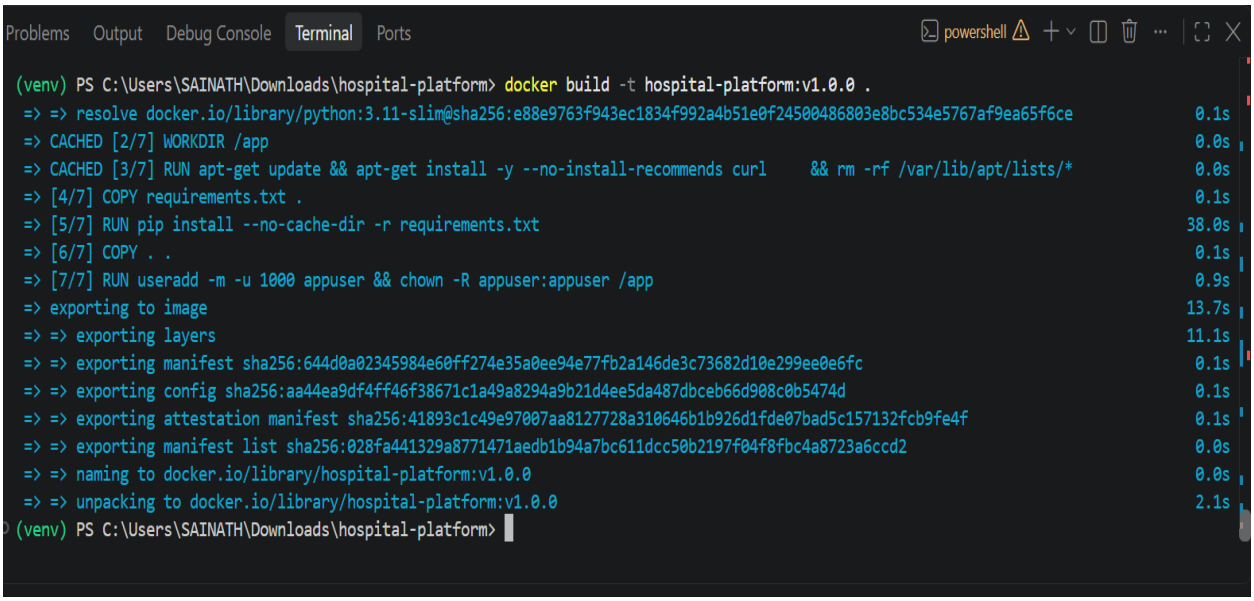
Step 5: Automated Testing and Code Coverage Analysis

- The project dependencies were defined in `requirements.txt`, including Flask, SQLAlchemy, PyJWT, Prometheus client, pytest, pytest-cov, and Flake8. The `pytest` command executed the automated test suite, with 12 tests passed and approximately 94% total code coverage. The test results were also exported to `reports/test-results.xml` for reporting.



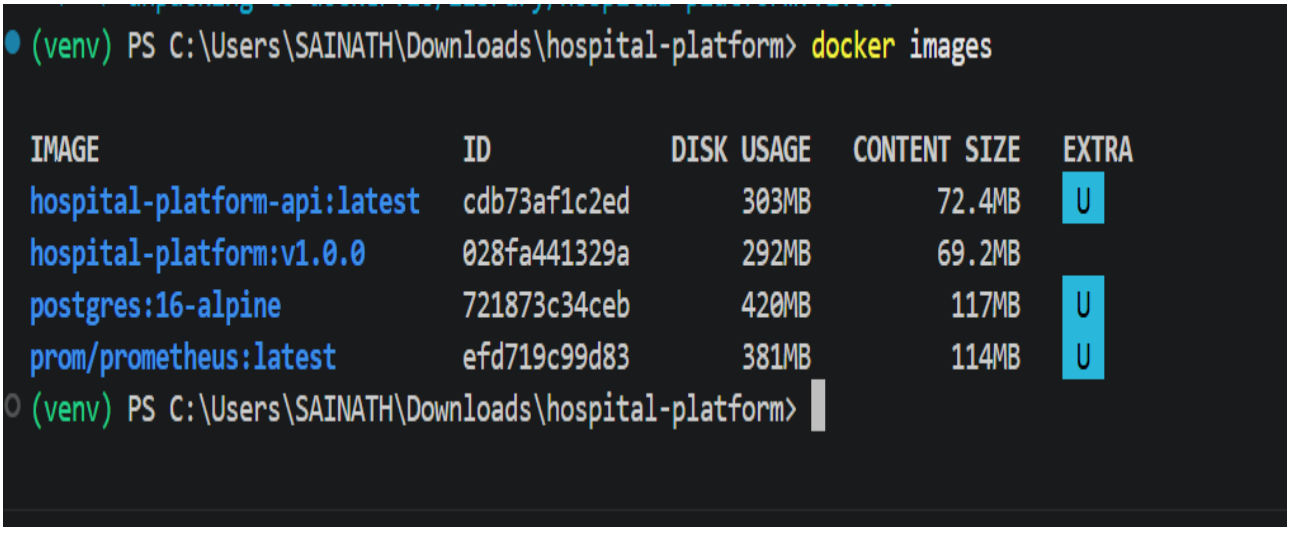
Step 6: Docker Image Creation and Version Tagging

- The command `docker build -t hospital-platform:v1.0.0 .` was executed to build the Hospital Platform Docker image using the Dockerfile. The build completed successfully, installing the required Python dependencies, copying the application files, and configuring a non-root user. The image was tagged as `hospital-platform:v1.0.0` for deployment.



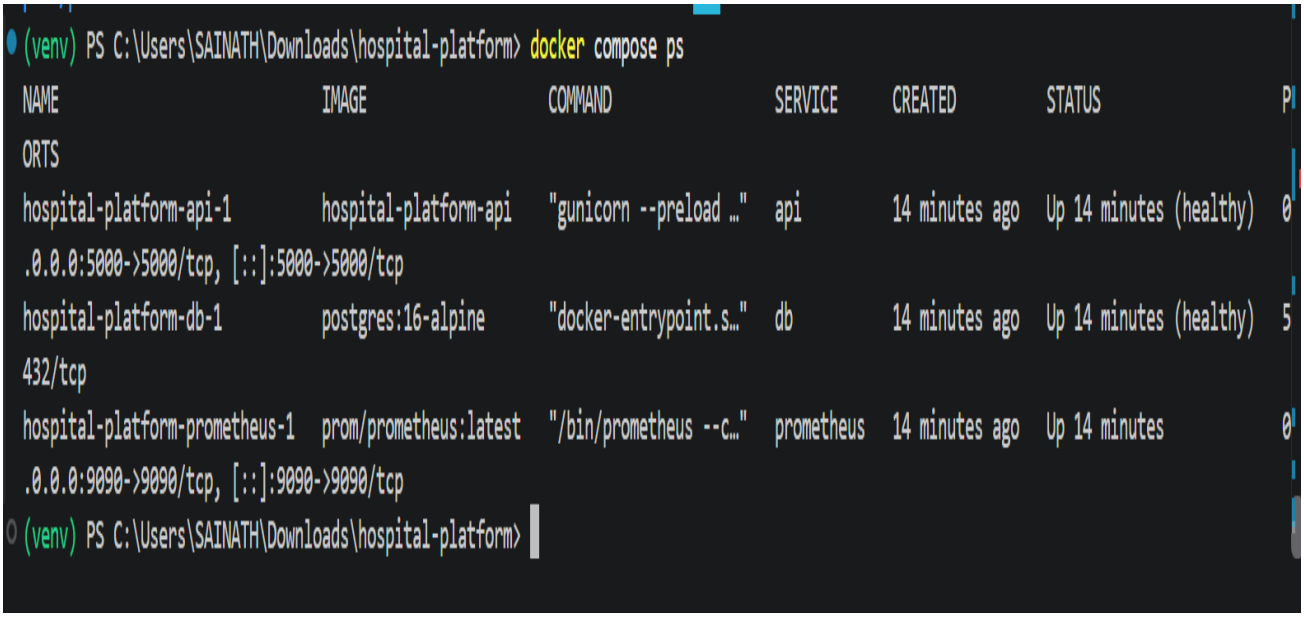
Step 7: Docker Image Verification

- The `docker images` command was executed to verify the locally available Docker images. The output lists the Hospital Platform API image, the versioned `hospital-platform:v1.0.0` image, PostgreSQL 16 Alpine, and Prometheus. This confirms that the application and its supporting service images are available locally for container deployment.



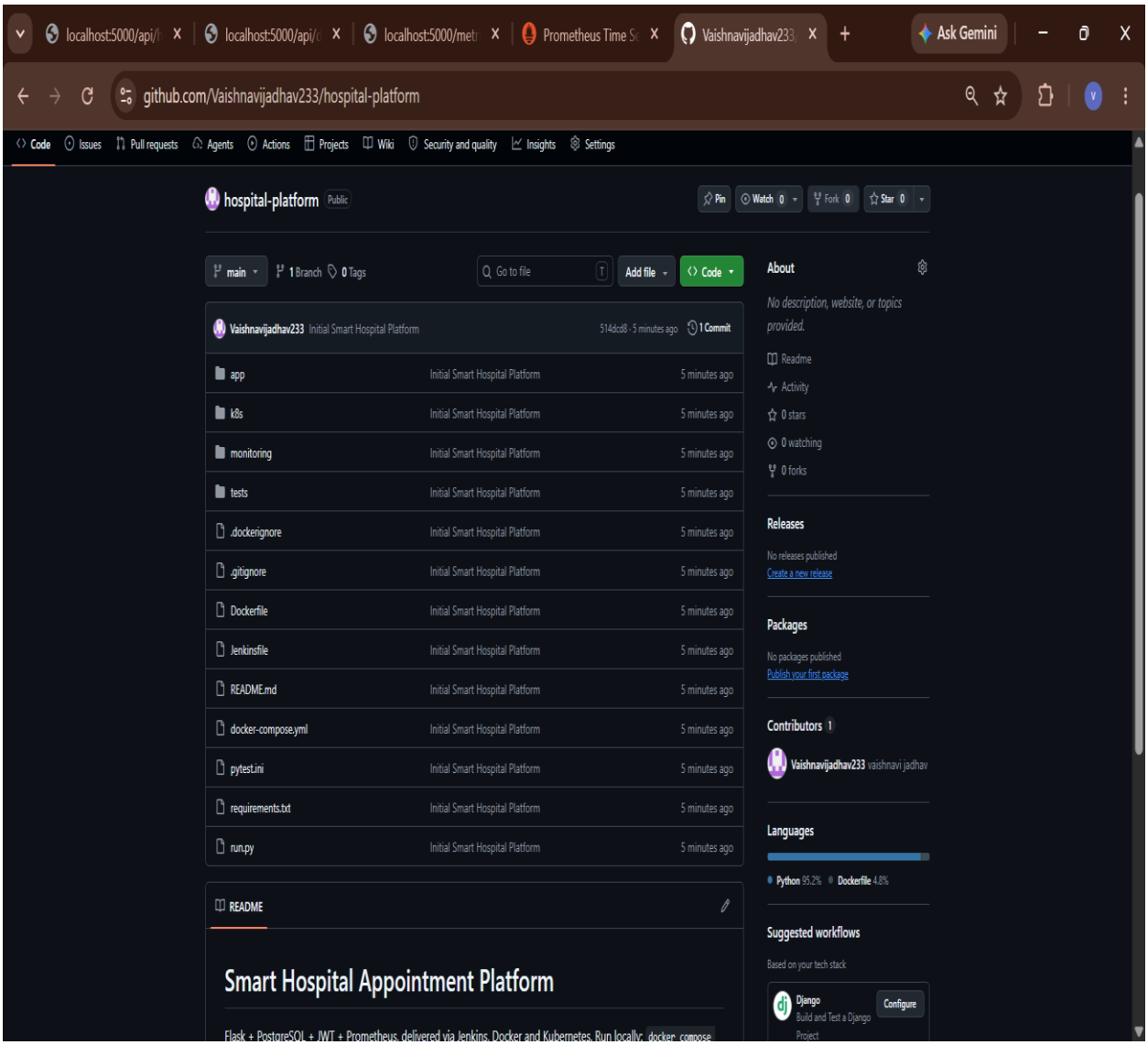
Step 8: Docker Compose Container Status Verification

- The `docker compose ps` command was executed to check the status of the Hospital Platform services. The output confirms that the API and PostgreSQL database containers are running and marked healthy, while the Prometheus monitoring container is running. The API is exposed on port `5000`, and Prometheus is accessible on port `9090`.



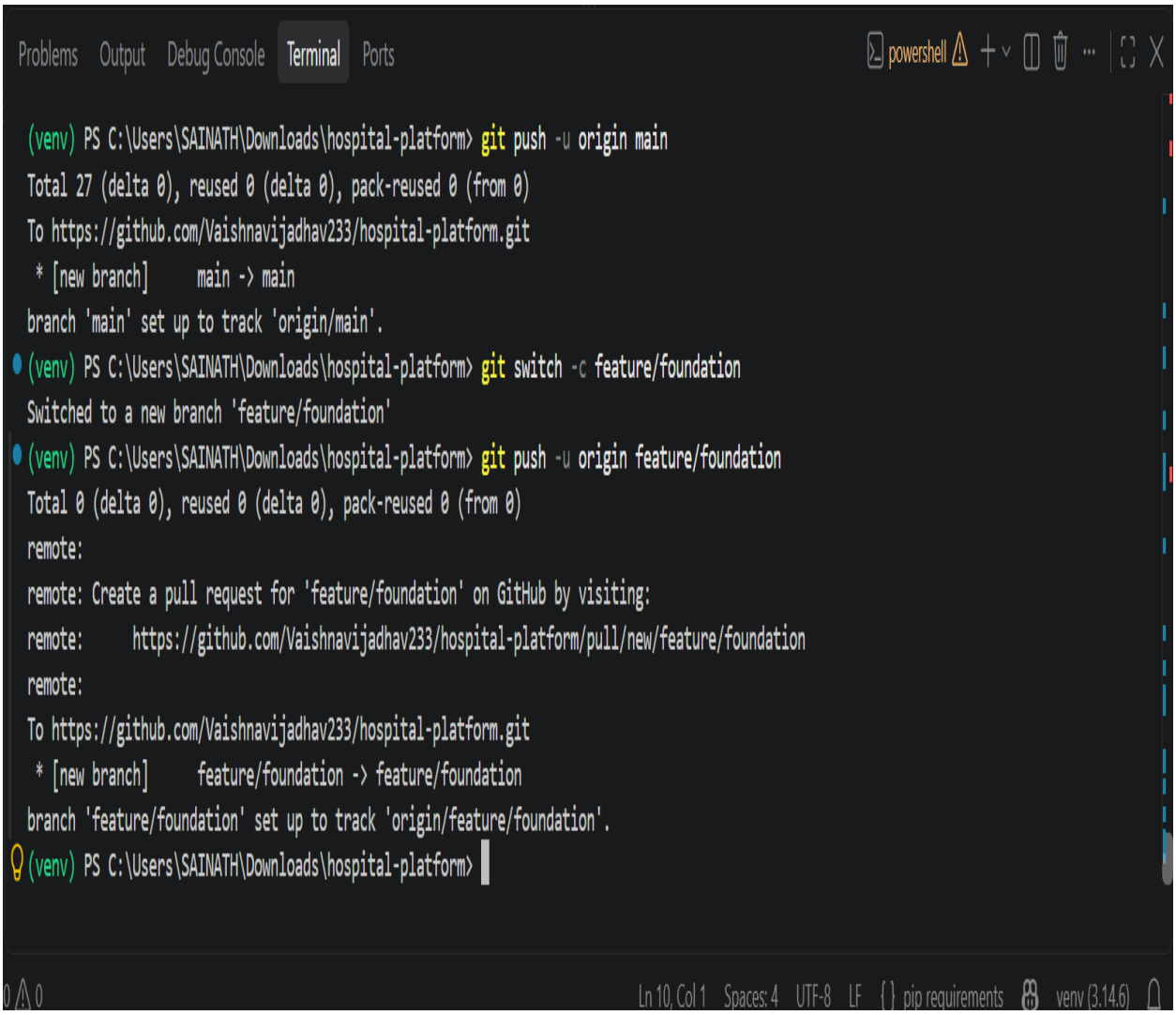
Step 9: GitHub Repository Setup and Source Code Upload

- The Hospital Platform project was uploaded to the public GitHub repository `Vaishnavijadhav233/hospital-platform`. The repository contains the application source code, Kubernetes manifests, monitoring configuration, automated tests, Dockerfile, Jenkinsfile, Docker Compose configuration, dependency list, and README.md. This enables source code version control and integration with the Jenkins CI/CD pipeline.



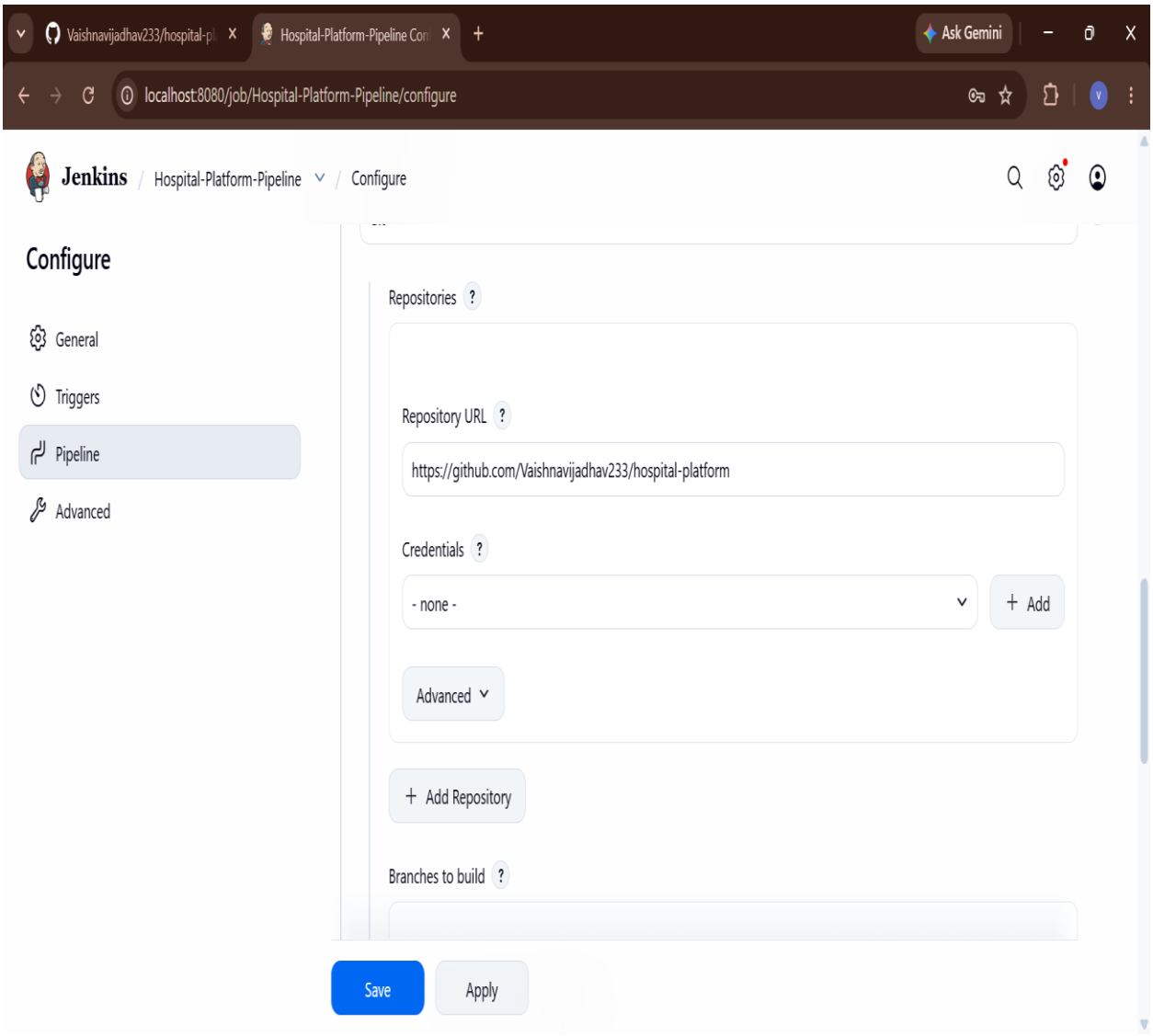
Step 10: Git Branch Management and Remote Repository Push

- The `git push -u origin main` command uploaded the project's main branch to GitHub. A new branch named `feature/foundation` was then created using `git switch -c` and pushed to the remote repository with `git push -u origin feature/foundation`. Both branches were configured to track their corresponding remote branches, supporting version control and feature-based development.



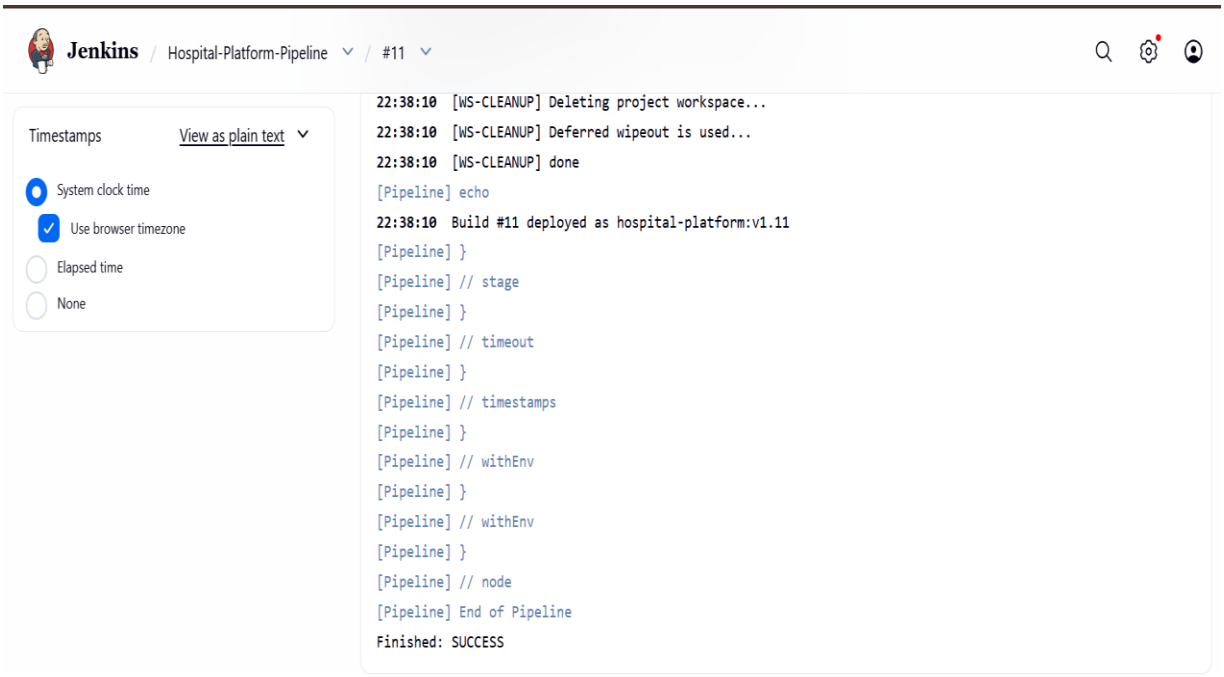
Step 11: Jenkins Pipeline Configuration with GitHub Integration

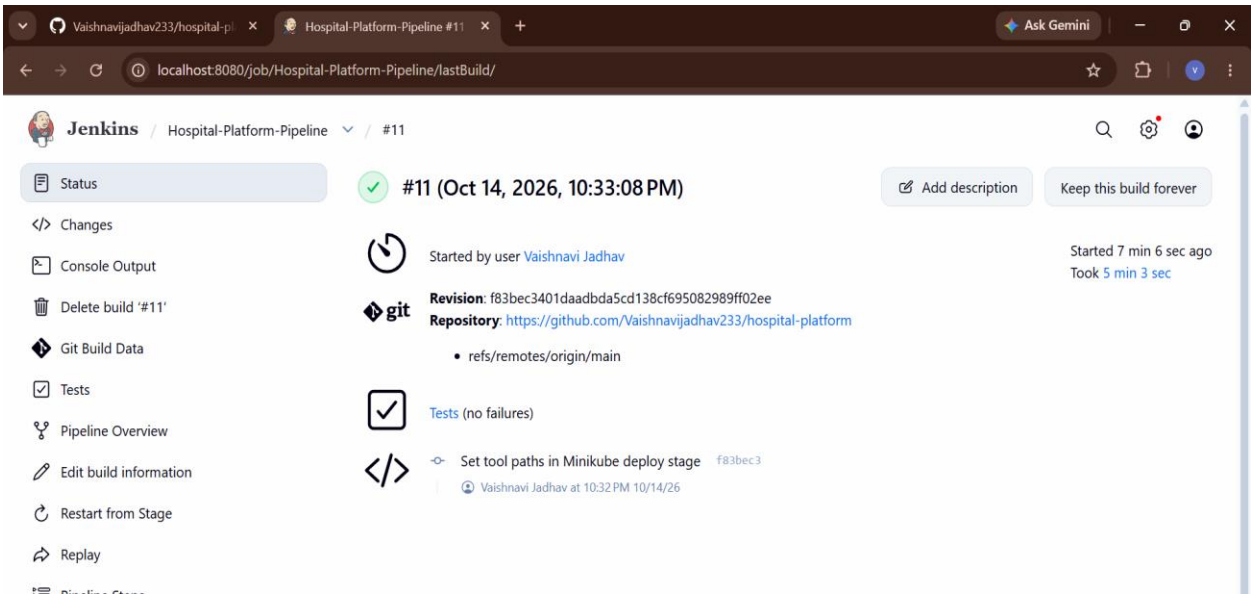
- The Jenkins job `Hospital-Platform-Pipeline` was configured to connect to the GitHub repository `Vaishnavijadhav233/hospital-platform`. The repository URL was entered in the pipeline configuration to enable Jenkins to retrieve the project source code for automated builds. The credentials field is set to “none” in the screenshot.



Step 12: Jenkins CI/CD Pipeline Execution and Successful Deployment

- Jenkins Build #11 completed successfully, as confirmed by the console output displaying `Finished: SUCCESS`. The pipeline reported that the application was deployed as `hospital-platform:v1.11`, indicating successful completion of the configured build and deployment stages.

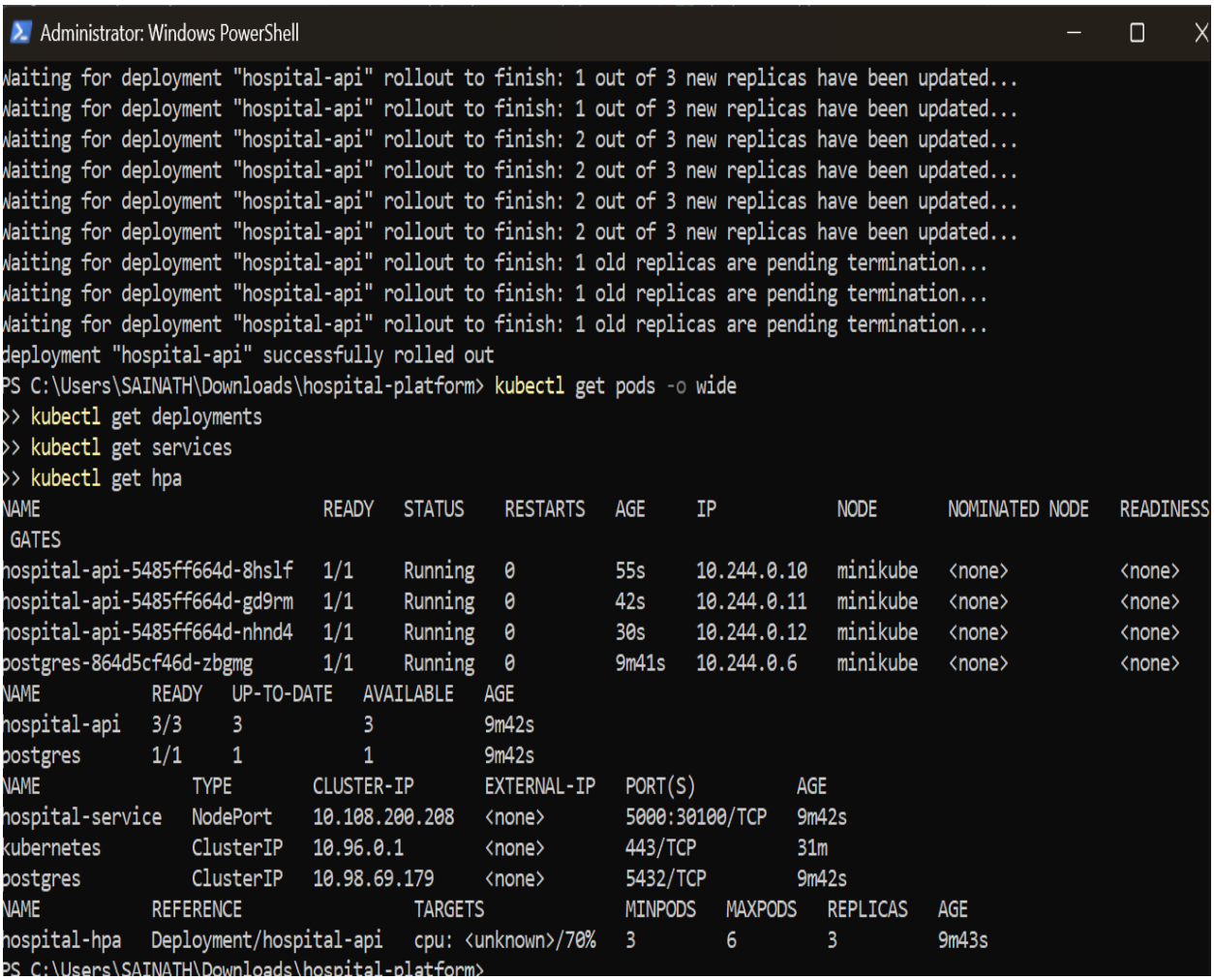




Step 13: Kubernetes Deployment Verification and Horizontal Pod Autoscaler Configuration

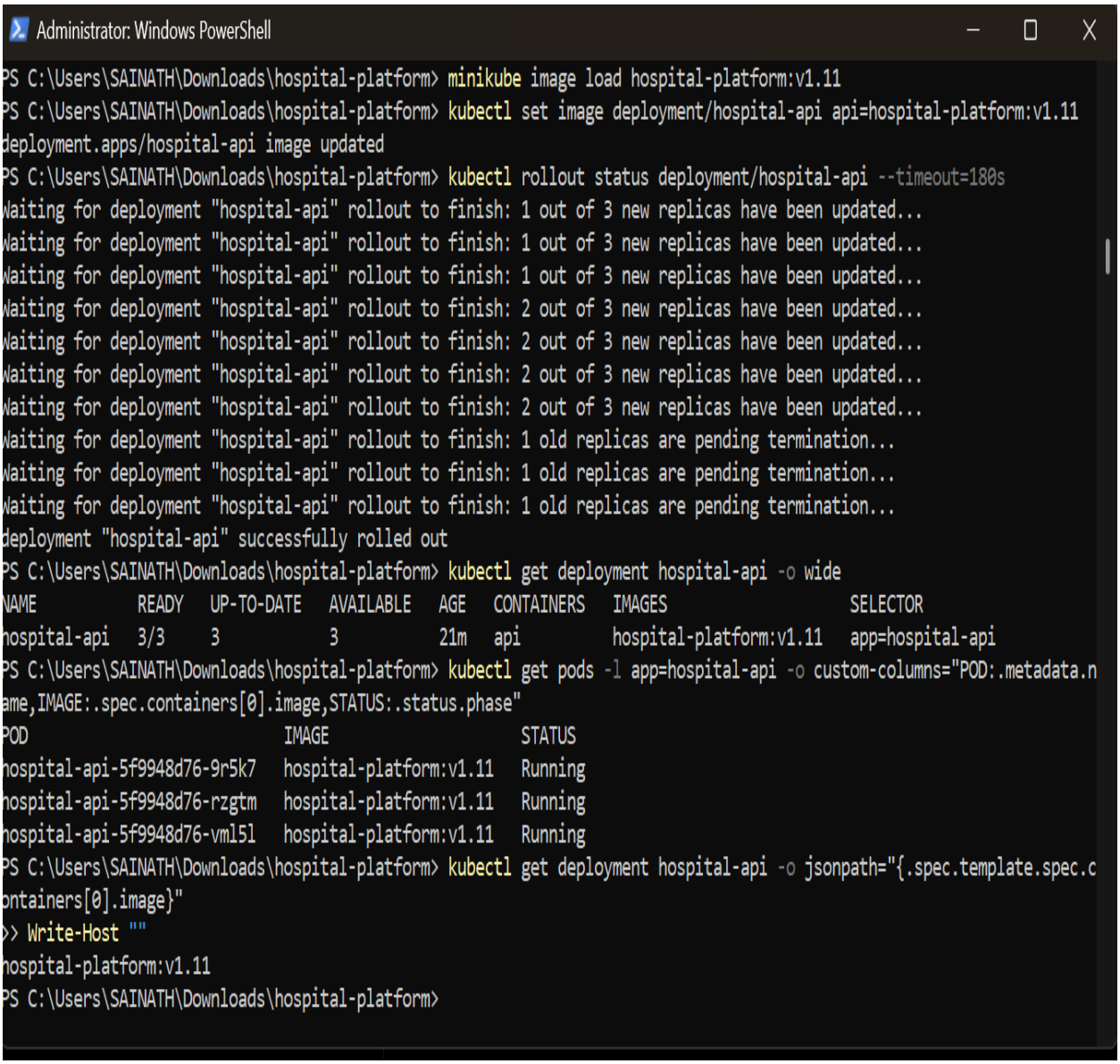
- The Kubernetes rollout completed successfully, and the `kubectl` commands verified three running API replicas and one running PostgreSQL pod. The `hospital-service` NodePort was configured on port `30100`. The Horizontal Pod Autoscaler (`hospital-hpa`) was also verified with a

minimum of 3 and maximum of 6 replicas and a CPU utilization target of 70%; however, the CPU metric was displayed as unknown.



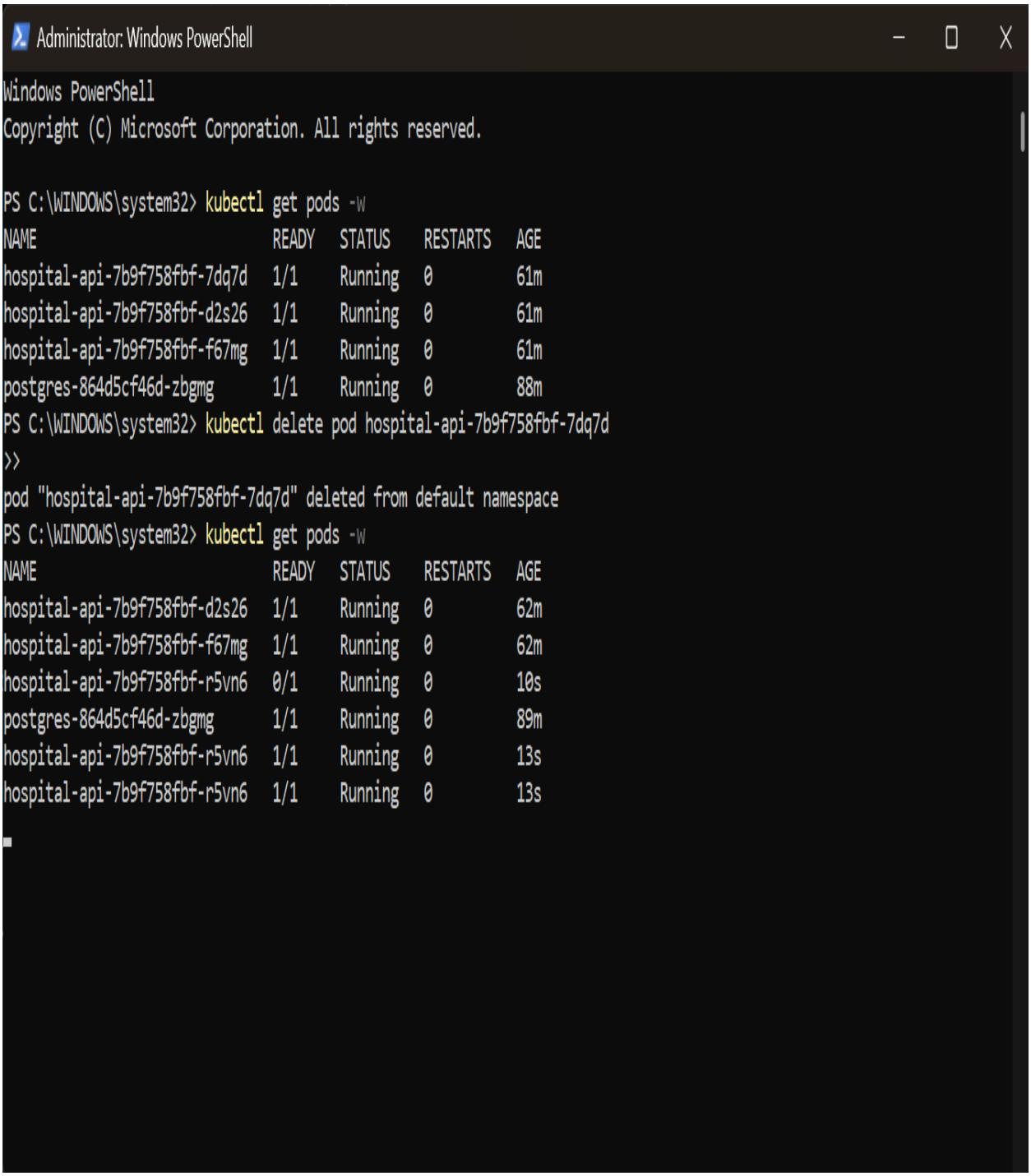
Step 14: Kubernetes Image Update and Rolling Deployment Verification

- The Docker image `hospital-platform:v1.11` was loaded into Minikube and applied to the `hospital-api` Deployment using `kubectl set image`. The rolling update completed successfully, and verification confirmed that all three API pods were running the updated image `v1.11`.



Step 15: Kubernetes Pod Self-Healing and Automatic Recovery

- The command `kubectl delete pod` was used to delete one running Hospital API pod intentionally. The `kubectl get pods -w` command then displayed Kubernetes creating a replacement pod automatically through the Deployment controller, restoring the desired three API replicas. This demonstrates the self-healing capability of Kubernetes.



*Github Repository Link: https://github.com/Vaishnavijadhav233/hospital-platform